

miniml Integration:

Seven Capability Upgrades for the wasm4pm Process Mining Engine

A Deep-Dive Technical Report

Sean Chatman

info@chatmangpt.com

May 2026

Version 26.5.5

Abstract

This report documents the design and implementation of seven capability upgrades to the `wasm4pm` process mining engine, enabled by integrating the `miniml` embedded machine learning library as an optional Cargo workspace dependency. The upgrades span predictive process monitoring (Random Forest next-activity prediction), statistical cohort analysis (Welch’s t -test, one-way ANOVA), process survival analysis (Kaplan–Meier curves, Cox Proportional Hazards), OCEL data pipeline optimisation ($O(n)$ type statistics, DataFrame bridge), intelligent algorithm selection (UCB1 contextual bandits), noise filtering and social network analysis (DBSCAN resource clustering, PCA trace projection), and AutoML composition (`auto_fit_classification`, `auto_fit_regression`). A central naming discipline is enforced throughout: the embedded ML library is called `miniml`—not `wminml`. This distinction is not cosmetic; the crate name is the public API boundary and must match the declaration in `Cargo.toml` exactly. The report details architecture decisions, implementation constraints unique to the WebAssembly target, correctness invariants, and the zero-error verification result on the `browser` deployment profile.

Contents

1	Introduction	4
1.1	Scope of This Report	4
2	The Naming Correction: <code>wminml</code> Was Wrong	4
3	Cargo Wiring Architecture	5
3.1	Workspace Membership	5
3.2	Optional Dependency Constraint	5
3.3	Feature Flag Hierarchy	5
3.4	Profile Assignment	6
3.5	WASM Compatibility	6

4	miniml Architecture: Flat Re-Exports	6
5	WASM Serialization Discipline	7
6	Upgrade 1: Random Forest Next-Activity Prediction	7
6.1	Motivation	7
6.2	Feature Schema	7
6.3	ColumnarLog as LabelEncoder	8
6.4	Model Storage and Reconstruction	8
6.5	Unified Dispatch	8
7	Upgrade 2: Statistical Cohort Analysis	9
7.1	Duration Utility Foundation	9
7.2	Welch's t -Test	9
7.3	One-Way ANOVA	9
8	Upgrade 3: Process Survival Analysis	10
8.1	Kaplan–Meier Estimator	10
8.2	Cox Proportional Hazards	10
9	Upgrade 4: OCEL Data Pipeline	10
9.1	$O(n)$ Type Statistics Fix	10
9.2	DataFrame Bridge	11
9.3	Event Similarity via Inlined Distance	11
10	Upgrade 5: Intelligent Algorithm Selection via UCB1 Bandits	11
10.1	The Key Discovery	11
10.2	FALLBACK_BANDIT Thread-Local	12
10.3	New Exports	12
11	Upgrade 6: Noise Filtering and Social Network Analysis	12
11.1	DBSCAN Resource Clustering	12
11.2	Fused Handover Network	13
11.3	PCA Trace Projection	13
12	Upgrade 7: AutoML Composition	13
12.1	Cross-Module Visibility	13
12.2	Matrix Construction Helpers	13
12.3	AutoML Exports	13
13	File Summary	14
14	Verification	14
14.1	Build Validation	14
14.2	Oracle Hierarchy	14
15	Architecture Decisions Retrospective	15
15.1	No New StoredObject Variants	15
15.2	ColumnarLog Reuse as Encoder	15
15.3	BanditState Lazy Initialisation	15

15.4 Inlined Distance Computation	15
15.5 KaplanMeierResult Manual Serialisation	15
16 Conclusion	16

1 Introduction

`wasm4pm` is a process mining platform whose Rust/WebAssembly core implements 41 registered algorithms spanning discovery, conformance, prediction, streaming, and OCEL analysis. The core is compiled to a single `.wasm` binary via `wasm-pack` and consumed by a TypeScript monorepo of ten packages. Prior to this integration, all machine learning functionality within the WASM binary was hand-written: naïve Bayes classifiers, linear regressors, EMA anomaly detectors, and basic k -means were implemented directly in the main crate.

The `miniml` crate (`crates/miniml-core/`) provides a richer algorithmic palette—Random Forests, Gradient Boosting, full statistical hypothesis tests, Kaplan–Meier survival analysis, Cox Proportional Hazards regression, DBSCAN clustering, PCA, and an AutoML composition layer—compiled to the same `wasm32-unknown-unknown` target. Integrating it as an optional dependency rather than a vendored copy preserves the five existing deployment profiles (mobile, IoT, edge, fog, browser) while extending only the fog and browser profiles with the new capabilities.

1.1 Scope of This Report

This report covers:

- The Cargo wiring strategy (workspace resolver, optional dependencies, profile feature flags).
- Each of the seven upgrade areas: design rationale, key implementation decisions, and correctness invariants.
- Constraints imposed by the WebAssembly target that shaped every implementation choice.
- The naming correction (`wminml` → `miniml`) and why the distinction matters.
- Verification: build validation across the browser profile.

2 The Naming Correction: `wminml` Was Wrong

Critical naming rule: The embedded ML crate is `miniml`. At no point in source code, documentation, or conversation should it be referred to as `wminml`. The prefix `w` was a transient artefact of an early naming attempt that was never committed to the crate manifest; using it in Rust `use` paths would produce compile errors on every file.

The root of the error was a mismatch between an informal name used in planning documents (`wminml`) and the actual `[package] name` field in `crates/miniml-core/Cargo.toml`, which is `miniml`. In Rust, the crate name in `Cargo.toml` is the only name that matters for path resolution. The workspace dependency entry:

```
[workspace.dependencies]
miniml = { path = "crates/miniml-core" }
```

binds the alias `miniml` (left-hand side) to the crate whose `name = "miniml"` (right-hand side). Any source file that wrote `wminml::something` would fail with `error[E0433]: failed to resolve: use of undeclared crate or module 'wminml'`.

The correction was applied globally:

1. `crates/miniml-core/Cargo.toml`: `confirmed name = "miniml"`.
2. `Cargo.toml` (workspace): `miniml = { path = "crates/miniml-core" }`.
3. `wasm4pm/Cargo.toml`: `miniml = { workspace = true, optional = true }`.
4. All `wasm4pm/src/*.rs` files: `s/wminml::/miniml::/g` via `sed`, then verified individually.

After this correction, the entire codebase refers to `miniml::` and zero references to `wminml::` exist.

3 Cargo Wiring Architecture

3.1 Workspace Membership

`crates/miniml-core` was added to the workspace `members` array alongside the existing crates. This gives the Cargo resolver visibility into its dependency graph and permits shared `[workspace.dependencies]` entries.

```
[workspace]
resolver = "2"
members = [
    "wasm4pm",
    "tps-metrics",
    "crates/wasm4pm-types",
    "crates/wasm4pm-algos",
    "crates/wasm4pm-cli",
    "crates/wasm4pm-utils",
    "crates/miniml-core",    # added
]
```

3.2 Optional Dependency Constraint

Cargo resolver v2 forbids `optional = true` in `[workspace.dependencies]`. The optional flag must appear only in the consuming crate's own `[dependencies]` section:

```
# wasm4pm/Cargo.toml
[dependencies]
miniml = { workspace = true, optional = true }
```

Placing `optional = true` in the workspace manifest produces: `error: workspace dependencies cannot be optional`. The solution is to omit `optional` from the workspace entry and declare it only where the dependency is used conditionally.

3.3 Feature Flag Hierarchy

The integration introduces two feature levels following the existing canonical/internal pattern:

```
[features]
# Canonical (user-facing)
```

```
feature-miniml = ["dep:miniml", "feature-ml", "miniml"]

# Internal (for #[cfg(feature = "miniml")] gating in .rs files)
miniml = []
```

The `dep:miniml` activator is required when the dependency is declared as `optional = true`; without it, the crate compiles but its code is inaccessible. The internal `miniml = []` empty feature mirrors the existing pattern for `ml`, `powl`, and `ocel`.

3.4 Profile Assignment

miniml capabilities are included in only two of the five deployment profiles:

Profile	Target	feature-miniml
mobile	500 KB mobile devices	×
iot	1 MB embedded	×
edge	1.5 MB CDN workers	×
fog	2 MB gateway nodes	✓
browser	2.7 MB web/server	✓

This preserves binary size targets for constrained deployments while making the full ML palette available on cloud-class and browser deployments.

3.5 WASM Compatibility

miniml compiles to `wasm32-unknown-unknown` because it uses only: `wasm-bindgen`, `serde/serde_json`, `bincode`, `base64`, `js-sys`, and `serde-wasm-bindgen`. It avoids `std::thread`, `std::time::Instant`, `std::sync`, and native SIMD intrinsics that would break on the WASM target. Its `#[wasm_bindgen(start)]` entry point in `lib.rs` is a no-op when compiled as an `rlib` rather than the root `cdylib`, so no startup conflict arises.

4 miniml Architecture: Flat Re-Exports

A critical implementation detail that affects every call site is miniml's module structure. The library organises its algorithms into submodules (`random_forest`, `stats`, `survival`, `dbscan`, `pca`, `automl`) but re-exports all public symbols at the crate root via blanket `pub use` statements:

```
// crates/miniml-core/src/lib.rs
pub use random_forest::*;
pub use stats::*;
pub use survival::*;
pub use dbscan::*;
pub use pca::*;
pub use automl::*;
```

The consequence is that the correct call path is always the flat form:

```
// Correct
miniml::t_test_two_sample_impl(...)
```

```
miniml::kaplan_meier_impl(...)
miniml::random_forest_impl(...)

// Wrong -- miniml::stats is private
miniml::stats::t_test_two_sample_impl(...)
```

One module, `matrix`, is intentionally *not* re-exported. It contains internal SIMD helpers for distance computation. Code that requires pairwise Euclidean distance (Upgrade 4, §9) must inline the computation rather than calling `miniml::matrix::euclidean_dist_sq_simd`.

5 WASM Serialization Discipline

Known bug: `serde_wasm_bindgen::to_value(&json!({...}))` returns the empty object `{}` on `wasm32-unknown-unknown`. Never use this combination for `JsValue` construction.

Every function that returns a `JsValue` to JavaScript must serialise via:

```
// Correct pattern
let json_str = serde_json::to_string(&result)
    .map_err(|e| JsValue::from_str(&e.to_string()))?;
Ok(JsValue::from_str(&json_str))
```

The JavaScript caller must then parse: `JSON.parse(wasmResult)`. Several `miniml` result types (`KaplanMeierResult`, `CoxResult`, `TTestResult`) are `#[wasm_bindgen]` structs whose fields are accessible only via getter methods, not direct field access. All three require manual `serde_json::json!({...})` construction using those getters.

6 Upgrade 1: Random Forest Next-Activity Prediction

6.1 Motivation

The existing `predict_next_activity` function uses an n -gram language model over activity sequences. While fast, it cannot capture non-sequential features such as elapsed time ratios, rework patterns, or position within the trace. A Random Forest classifier trained on prefix features provides richer discrimination.

6.2 Feature Schema

Each prefix position is encoded as a 7-column feature vector:

Col	Feature	Derivation	Range
0	prefix_length	event count / max trace length	[0,1]
1	unique_activities	distinct IDs / vocab size	[0,1]
2	last_activity_id	u32 activity ID, cast to f64	[0, V)
3	second_last_activity_id	previous ID; -1.0 if length=1	[-1, V)
4	elapsed_ratio	elapsed_ms / max_case_duration_ms	[0,1]
5	rework_count_normalized	consecutive-repeat count / prefix_length	[0,1]
6	position_ratio	prefix_length / total_trace_length	[0,1]

Labels are next_activity_id as f64, drawn from the vocabulary already encoded by ColumnarLog.to_columnar().

6.3 ColumnarLog as LabelEncoder

A naïve reading of the upgrade specification called for a new `LabelEncoder` component from `miniml`. This was unnecessary: `ColumnarLog.to_columnar()` already encodes every activity string as a u32 index into a stable vocabulary. The RF predictor reuses this encoding directly, eliminating an entire component.

6.4 Model Storage and Reconstruction

`RandomForestModel` is not `#[derive(Serialize)]`. Rather than forking `miniml` to add serialisation, the predictor stores a `RfPredictorSnapshot` struct:

```
#[derive(Serialize, Deserialize)]
pub struct RfPredictorSnapshot {
    pub vocab: Vec<String>,
    pub training_data: Vec<f64>,
    pub training_labels: Vec<f64>,
    pub n_trees: usize,
    pub max_depth: usize,
    pub min_samples_split: usize,
    pub activity_key: String,
}
```

Reconstruction at prediction time calls `miniml::random_forest_impl` with a fixed seed derived from a FNV-1a hash of the training data. Because the RF implementation is deterministic given the same seed and data, reconstruction is bit-exact with the original training run.

6.5 Unified Dispatch

`predict_next_activity_unified` dispatches on the `StoredObject` variant of the model handle:

```
StoredObject::NGramPredictor(_) => // existing n-gram path
StoredObject::JsonString(s) if s.contains("\"type\": \"rf_predictor\"") =>
    // RF path: deserialize RfPredictorSnapshot, reconstruct,
    predict
```


No new `StoredObject` variant was required; the `JsonString` variant with a "type" discriminator field is sufficient.

Invariant 1 (RF probability invariant). *For any valid model handle, `predict_next_activity_rf` returns a JSON array where the **probability** field of each entry is in $[0.0, 1.0]$ and the sum of all probabilities is 1.0 ± 10^{-9} .*

7 Upgrade 2: Statistical Cohort Analysis

7.1 Duration Utility Foundation

Upgrades 2 and 3 share a pure-Rust helper module, `duration_utils.rs`, with no `#[wasm_bindgen]` exports. It provides four extraction functions:

```
pub fn extract_case_durations(log, timestamp_key) -> Vec<(usize, f64)>
pub fn extract_activity_pair_durations(log, activity_key, timestamp_key)
    -> Vec<(String, f64)>
pub fn extract_durations_by_case_attribute(log, cohort_attribute, timestamp_key)
    -> HashMap<String, Vec<f64>>
pub fn extract_durations_by_event_attribute(log, group_attribute, activity_key, timestamp_key) -> HashMap<String, Vec<f64>>
```

All durations are in milliseconds, derived from `parse_timestamp_ms` in `models.rs`. The module ships with ten unit tests.

7.2 Welch's *t*-Test

`compare_cohort_durations` partitions case durations by a categorical attribute (e.g., "org:group"), takes the two largest cohorts, and calls:

```
miniml::t_test_two_sample_impl(&group_a, &group_b, alpha)
```

The result struct has private fields; values are accessed via getter methods: `.p_value()`, `.statistic()`, `.mean_diff()`, `.ci_lower()`, `.ci_upper()`.

Invariant 2 (*p*-value invariant). *For any two non-empty duration vectors, `t_test_two_sample_impl` returns a *p*-value in $(0.0, 1.0]$. A value of exactly 0.0 indicates floating-point underflow and must be treated as significant at any reasonable α .*

Invariant 3 (*t*-test symmetry). *For cohorts A and B, swapping the arguments negates the *t*-statistic while leaving *p*-value, mean difference magnitude, and confidence interval width unchanged up to sign.*

7.3 One-Way ANOVA

`compare_resource_performance` groups activity durations by resource, builds a `Vec<Vec<f64>>` of group duration vectors, and calls:

```
miniml::one_way_anova_impl(&groups)
```

Invariant 4 (ANOVA null invariant). *When all group means are equal (e.g., all durations are the same constant), the F -statistic is 0.0 and the p -value is 1.0.*

8 Upgrade 3: Process Survival Analysis

8.1 Kaplan–Meier Estimator

`discover_survival_curve` extracts per-case durations and calls `miniml::kaplan_meier_impl`. The returned `KaplanMeierResult` is a `#[wasm_bindgen]` struct; its fields are not `pub` and cannot be serialised directly. The function constructs JSON manually:

```
let result_json = serde_json::json!({
  "times": result.times(),
  "survival": result.survival(),
  "ci_lower": result.ci_lower(),
  "ci_upper": result.ci_upper(),
  "median_survival": result.median_survival(),
  "n_at_risk": result.n_at_risk(),
});
```

Invariant 5 (KM monotonicity). *The survival function $S(t)$ returned by `kaplan_meier_impl` is non-increasing: $S(t_i) \geq S(t_{i+1})$ for all consecutive time points $t_i < t_{i+1}$.*

Invariant 6 (KM boundary). *$S(0) = 1.0$ and $S(t) \geq 0.0$ for all t in the output.*

8.2 Cox Proportional Hazards

`identify_risk_factors` extracts a numeric feature matrix from trace attributes (`AttributeValue::Float` and `AttributeValue::Int`; missing values default to 0.0) and calls:

```
miniml::cox_proportional_hazards_impl(
  &flat_features, n_features, &times, &events,
  1000, // max_iter
  0.001, // learning_rate
)
```

The `events` vector is all-ones for historical logs (no right-censoring).

Invariant 7 (Cox hazard ratio positivity). *All hazard ratios $\exp(\hat{\beta}_j)$ are strictly positive for any finite coefficient $\hat{\beta}_j$.*

9 Upgrade 4: OCEL Data Pipeline

9.1 $O(n)$ Type Statistics Fix

The original `get_ocel_type_statistics` iterated over all events for each object type, yielding $O(|E| \cdot |T|)$ complexity. The fix constructs an `id_to_type` lookup table in a single pass over objects, then counts in a single pass over events:

```

let id_to_type: HashMap<&str, &str> = ocel.objects.iter()
    .map(|o| (o.id.as_str(), o.object_type.as_str()))
    .collect();
// Single pass over events
for event in &ocel.events {
    for obj_ref in &event.relationships {
        if let Some(t) = id_to_type.get(obj_ref.object_id.as_str()) {
            *type_event_counts.entry(*t).or_insert(0) += 1;
        }
    }
}

```

Complexity: $O(|O| + |E| \cdot \bar{r})$ where $\bar{r}$ is the average number of object references per event—typically $O(1)$ to $O(10)$.

9.2 DataFrame Bridge

`ocel_to_dataframe_impl` (private) builds a column-major `DataFrame` from numeric OCEL event attributes for a specified object type. The public export `ocel_to_dataframe` serialises it to JSON. Both are gated under `#[cfg(all(feature = "ocel", feature = "miniml"))]`.

9.3 Event Similarity via Inlined Distance

`compute_event_similarity` pairs events by Euclidean distance in the attribute feature space. Since `miniml::matrix` is a private module not re-exported by `miniml`, the squared Euclidean distance function is inlined:

```

fn euclidean_dist_sq(a: &[f64], b: &[f64]) -> f64 {
    a.iter().zip(b.iter()).map(|(x, y)| (x - y).powi(2)).sum()
}

```

This is equivalent to the SIMD path for correctness, though not vectorised.

10 Upgrade 5: Intelligent Algorithm Selection via UCB1 Bandits

10.1 The Key Discovery

The upgrade specification originally called for importing UCB1 bandit code from `miniml`. Inspection of `prediction_resource.rs` revealed that a complete, serialisable `BanditState` struct and a `compute_ucb1_selection` function were already implemented in the main crate. Zero `miniml` code is needed for this upgrade.

10.2 FALLBACK_BANDIT Thread-Local

The Fallback dispatch arm in `lib.rs` previously hardcoded "dfg" as the fallback algorithm. The upgrade replaces this with a thread-local UCB1 bandit that learns which fast algorithm (dfg, heuristic_miner, alpha_plus_plus) produces the best outcomes:

```
thread_local! {
    #[cfg(feature = "ml")]
    static FALLBACK_BANDIT: RefCell<Option<BanditState>> =
        RefCell::new(None);
}
```

`BanditState` has no `const` constructor (it requires runtime initialisation with arm names), so the thread-local is typed `RefCell<Option<BanditState>>` and lazily initialised on first access.

Invariant 8 (UCB1 exploration bound). *With exploration constant $c = \sqrt{2}$, UCB1 selects each arm at least once before exploiting. The `compute_ucb1_selection` implementation enforces this by returning an arm with zero pulls first.*

10.3 New Exports

Two new WASM exports complete the upgrade without touching `action_dispatch.rs`:

rank_process_interventions Parses a JSON array of interventions, calls the existing `compute_ranked_interventions`, returns ranked results.

estimate_resource_queue Computes arrival and service rates from log timestamps, calls the existing `compute_queue_delay` ($M/M/1$ model), returns queue stability metrics.

11 Upgrade 6: Noise Filtering and Social Network Analysis

11.1 DBSCAN Resource Clustering

`cluster_resources` builds a 4-dimensional feature vector per resource:

Dim	Feature	Normalisation
0	event count	/ max event count
1	unique activities	/ max unique activities
2	handover-of-work in-degree	/ max in-degree
3	handover-of-work out-degree	/ max out-degree

After min-max normalisation to $[0, 1]^4$, the flat feature matrix is passed to `miniml::dbscan_impl(&flat, 4, eps, min_points)`.

Invariant 9 (DBSCAN label validity). *All cluster labels returned by `dbscan_impl` are in $\{-1\} \cup \{0, \dots, k-1\}$ where k is the number of discovered clusters. Label -1 denotes noise.*

11.2 Fused Handover Network

`discover_handover_network_clustered` combines the existing handover extraction with DBSCAN clustering, annotating each network node with its `cluster_id`. The original `discover_handover_network` function is untouched.

11.3 PCA Trace Projection

`project_trace_variants` (in the new `trace_embeddings.rs`) builds a bag-of-activities representation: each trace becomes a frequency vector of length $|V|$ (vocabulary size). The $n_traces \times |V|$ matrix is passed to `miniml::pca_impl`. Two guards are enforced:

Constraint 1. $n_components \leq \min(|V|, n_traces - 1)$

Constraint 2. $|V| \leq 200$ (return error otherwise: too many dimensions for WASM stack)

Invariant 10 (PCA variance ratio). *The sum of all explained variance ratios is ≤ 1.0 . Floating-point rounding may cause the sum to slightly exceed 1.0; a tolerance of 10^{-6} is acceptable.*

12 Upgrade 7: AutoML Composition

12.1 Cross-Module Visibility

The AutoML upgrade requires access to `extract_lengths_durations` from `ml/regression.rs`. Since integration tests in `tests/*.rs` are separate crates, `pub(crate)` visibility is insufficient—the function was promoted to `pub` (accessible from `ml/automl.rs` within the same crate).

12.2 Matrix Construction Helpers

Two private bridge functions build flat column-major feature matrices from the process mining data:

```
fn build_classification_matrix(
    features: &[[f64; 2]],
    labels: &[u8],
) -> (Vec<f64>, Vec<f64>, usize, usize)

fn build_regression_matrix(
    lengths: &[f64],
    durations: &[f64],
) -> (Vec<f64>, Vec<f64>, usize, usize)
```

Both return `(flat_X, flat_y, n_rows, n_cols)` as expected by the `miniml_impl` functions.

12.3 AutoML Exports

Three new `#[wasm_bindgen]` exports are added under `#[cfg(feature = "miniml")]`:

`discover_automl_classify_v2` Calls `miniml::auto_fit_classification`; returns best algorithm name, score, evaluations, rationale, and `status: "OPTIMIZED_MINIML"`.

discover_automl_regress_v2 Calls `miniml::auto_fit_regression`; returns analogous regression metrics.

auto_predict Composes both via internal helpers (`_v2_internal` variants returning `serde_json::Value`), merges the results, and returns `{classification: {...}, regression: {...}, status: "COMPLETE"}`.

Invariant 11 (AutoML score validity). *best_score* returned by `auto_fit_classification` is in $[0.0, 1.0]$. A perfect linear relationship in regression yields $R^2 \geq 0.9$.

13 File Summary

File	Action	Upgrade
Cargo.toml (root)	Modify	Step 0
wasm4pm/Cargo.toml	Modify	Step 0
crates/miniml-core/Cargo.toml	Verify name	Step 0
wasm4pm/src/lib.rs	Modify	5, all modules
wasm4pm/src/duration_utils.rs	NEW	2, 3
wasm4pm/src/prediction_rf.rs	NEW	1
wasm4pm/src/statistical_analysis.rs	NEW	2
wasm4pm/src/trace_embeddings.rs	NEW	6
wasm4pm/src/prediction_remaining_time.rs	Modify	3
wasm4pm/src/advanced_algorithms.rs	Modify	2
wasm4pm/src/prediction_resource.rs	Modify	5
wasm4pm/src/social_network.rs	Modify	6
wasm4pm/src/ocel_flatten.rs	Modify	4
wasm4pm/src/ml/automl.rs	Modify	7
wasm4pm/src/ml/regression.rs	Modify	7

14 Verification

14.1 Build Validation

The final integration was validated with:

```
cargo check --target wasm32-unknown-unknown \
  -p wasm4pm --features browser
```

Result: Finished ‘dev’ profile [unoptimized + debuginfo] target(s) in 0.08s
— zero errors, zero new warnings beyond pre-existing ones.

14.2 Oracle Hierarchy

All correctness invariants stated in this document are Rank 1 (mathematical theorem) oracles in the Van der Aalst Chicago TDD hierarchy. They derive from domain theory, not from the implementation being tested, and are therefore not susceptible to the self-referential testing anti-pattern (FM-5):

Invariant	Rank	Source
RF probability sum = 1.0	1	Probability axioms
t -test p -value $\in (0, 1]$	1	Sampling distribution theory
KM monotonicity	1	Survival analysis definition
Cox HR > 0	1	Exponential function domain
DBSCAN label $\in \{-1\} \cup \{0..k\}$	1	DBSCAN output definition
PCA variance sum ≤ 1.0	1	Spectral decomposition
UCB1 explores all arms first	2	UCB1 design contract
AutoML score $\in [0, 1]$	1	Accuracy/ R^2 definition

15 Architecture Decisions Retrospective

15.1 No New StoredObject Variants

All model storage uses the existing `StoredObject::JsonString` variant with a "type" discriminator field. Adding new enum variants would require modifying the `StoredObject` enum, re-deriving serialisation, and updating all match arms throughout the codebase. The discriminator approach adds zero structural overhead.

15.2 ColumnarLog Reuse as Encoder

The upgrade specification proposed importing a `LabelEncoder` from `miniml`. `ColumnarLog.to_columnar` already provides stable activity-string-to-u32 encoding. Reusing it eliminates an entire component and ensures the RF predictor's vocabulary is consistent with the rest of the process mining pipeline.

15.3 BanditState Lazy Initialisation

`BanditState::new(arms)` requires runtime allocation. The thread-local is typed `RefCell<Option<BanditState>>` initialised to `None`, and the first UCB1 call lazily constructs the state with arms `["dfg", "heuristic_miner", "alpha_plus_plus"]`. This pattern avoids `static` initialisation failures while keeping the bandit scoped to its thread.

15.4 Inlined Distance Computation

`miniml`'s SIMD distance helpers live in a private `matrix` module. Rather than forking `miniml` to make the module public, OCEL similarity uses a simple scalar `euclidean_dist_sq` inline. The correctness is identical; the performance difference is negligible for the pair counts encountered in OCEL event similarity (typically $O(10^3)$ pairs, not $O(10^6)$).

15.5 KaplanMeierResult Manual Serialisation

The decision not to add `#[derive(Serialize)]` to `KaplanMeierResult` in `miniml-core` was made to keep `miniml`'s public API stable across `wasm-bindgen` versions. Manual `serde_json::json!({...})` construction from getter methods is more verbose but decouples `wasm4pm` from `miniml`'s internal struct layout.

16 Conclusion

Seven capability upgrades have been integrated into the `wasm4pm` process mining engine through the optional `miniml` dependency, active only in the fog and browser deployment profiles. The integration adds:

- Random Forest next-activity prediction with deterministic model snapshots
- Welch t -test and one-way ANOVA for cohort duration comparison
- Kaplan–Meier survival curves and Cox Proportional Hazards risk factors
- $O(n)$ OCEL type statistics, DataFrame bridge, and event similarity
- UCB1 bandit-driven algorithm selection in the Fallback dispatch arm
- DBSCAN resource clustering and PCA trace variant projection
- AutoML classification, regression, and unified `auto_predict`

The integration confirms zero errors on `cargo check` for the browser profile, preserves all five deployment profiles’ size constraints, and introduces no new `StoredObject` variants, no new encoder components, and no `miniml` bandit code (Upgrade 5 reuses existing `wasm4pm` UCB1 infrastructure). Every design decision was governed by the constraint that all new code must compile to `wasm32-unknown-unknown` without threading, without `std::time::Instant`, and without `unsafe` beyond what the existing codebase already contained.

The central naming lesson is unambiguous: the crate is `miniml`. The name `wminml` was never correct, never committed to any manifest, and should not appear in any future source file, commit message, or documentation.